



39-MAVZU · 42 TADAN · V BOB. DRF VA DEPLOY

Django REST Framework (DRF) ga kirish.

Serializerlar va API View'lar

API nima, REST arxitekturasi, JSON formati, HTTP metodlari va status kodlari; DRF'ni o'rnatish, Serializer va ModelSerializer, validatsiya, @api_view va APIView, Response, brauzerda ko'rinadigan API.

2 soat · 1 nazariy + 1 amaliy

Bu darsda nimani o'rganasiz

- **API** nima va zamonaviy ilovalar nima uchun unga muhtojligini;
- **REST** arxitekturasi qoidalari va **JSON** formatini;
- Django REST Framework'ni o'rnatish va sozlashni;
- **Serializer** — Python obyektlarini JSON'ga va aksincha aylantirishni;
- API View'lar yozib, ularni brauzerda sinashni.

API nima?

IV bobda Django **HTML sahifalar** qaytardi — ularni faqat brauzer ko'rsata oladi. Lekin bugun bitta xizmatdan ko'plab "mijozlar" foydalanadi:

-  Mobil ilova (Android, iOS) — unga HTML kerak emas, faqat **ma'lumot** kerak;
-  React/Vue kabi frontend dasturlar — sahifani o'zi chizadi, serverdan faqat ma'lumot oladi;
-  Telegram bot, boshqa saytlar, aqlli soatlar...

API (Application Programming Interface — dasturlash interfeysi) — dasturlar bir-biri bilan "gaplashishi" uchun kelishilgan qoidalar. Veb-API — ma'lumotni **JSON** ko'rinishida beradigan URL'lar to'plami.



O'xshatish: restoran (yana!) va yetkazib berish

IV bobdagi sayt — restoran zali: taom chiroyli likopchada (HTML) beriladi. **API** — yetkazib berish xizmati: o'sha oshxona, o'sha taom, lekin konteynerda (JSON). Mijoz uni o'z uyida (mobil ilova, React sayt) o'zi xohlagan likopchaga soladi. Oshxona (back-end) bitta — mijozlar ko'p.



Mobil ilova



React sayt



Telegram bot

⇔ JSON ⇔



Django REST API

/api/maqolalar/



PostgreSQL

JSON — ma'lumot almashish tili

24-mavzuda tanishgan **JSON** (JavaScript Object Notation) — API'larning asosiy formati. U Python'dagi dict/list'ga va JavaScript'dagi obyekt/massivga juda o'xshaydi:

JSON

```
{
  "id": 5,
  "sarlavha": "Bilimlar bellashuvi",
  "chop_etilgan": true,
  "korishlar": 128,
  "kategoriya": { "id": 2, "nomi": "Tadbirlar" },
  "teglar": ["olimpiada", "8-sinf"],
  "rasm": null
}
```

Qoidalar: kalitlar **qo'sh qo'shtirnoqda**; qiymatlar — matn, son, `true` / `false`, `null`, obyekt `{}` yoki massiv `[]`.

REST arxitekturası

REST — API'ni tartibli va tushunarli loyihalash uslubi. Asosiy g'oya: hamma narsa — **resurs** (maqola, foydalanuvchi, mahsulot), har bir resursning o'z **URLi** bor, amallar esa **HTTP metodlari** bilan bildiriladi.

Amal (CRUD)	HTTP metod	URL	Javob kodi
Barcha maqolalar ro'yxati	GET	/api/maqolalar/	200 OK
Bitta maqola	GET	/api/maqolalar/5/	200 / 404
Yangi maqola yaratish	POST	/api/maqolalar/	201 Created
To'liq yangilash	PUT	/api/maqolalar/5/	200
Qisman yangilash	PATCH	/api/maqolalar/5/	200
O'chirish	DELETE	/api/maqolalar/5/	204 No Content

- URL'da **ot** (resurs), ko'plikda: `/api/maqolalar/` ✓, `/api/getMaqola/` ✗, `/api/maqola/ochir/5/` ✗ — amal metod orqali bildiriladi.
- Xatolar ham status kod bilan: `400` — noto'g'ri ma'lumot, `401` — tizimga kirilmagan, `403` — ruxsat yo'q, `404` — topilmadi.

DRF'ni o'rnatish

Django REST Framework (DRF) — Django'da API yaratish uchun eng mashhur kutubxona.

Terminal

```
pip install djangorestframework
pip freeze > requirements.txt
```

```
INSTALLED_APPS = [  
    # ...  
    "rest_framework",  
    "blog",  
]  
  
REST_FRAMEWORK = {  
    "DEFAULT_PERMISSION_CLASSES": [  
        "rest_framework.permissions.IsAuthenticatedOrReadOnly",    # o'qish –  
        hamмага, yozish – kirganlarga  
    ],  
}
```

Serializer — tarjimon

Bazadan olingan **Maqola** obyektini to'g'ridan-to'g'ri JSON qilib bo'lmaydi. **Serializer** ikki yo'nalishda ishlaydi:

- **Serializatsiya:** model obyekti → Python dict → JSON (javob yuborish uchun);
- **Deserializatsiya:** JSON → tekshirish (validatsiya) → model obyekti (kelgan ma'lumotni saqlash uchun).



Django Forms bilan o'xshashlik

Serializer — API uchun **Forma** (37-mavzu): u ham maydonlarni belgilaydi, ma'lumotni tekshiradi va saqlaydi. Farqi: forma HTML bilan, serializer JSON bilan ishlaydi. **Form** ↔

Serializer, **ModelForm** ↔ **ModelSerializer**.

blog/serializers.py (yangi fayl)

```
from rest_framework import serializers
from .models import Kategoriya, Maqola

class KategoriyaSerializer(serializers.ModelSerializer):
    class Meta:
        model = Kategoriya
        fields = ["id", "nomi", "slug"]

class MaqolaSerializer(serializers.ModelSerializer):
    muallif = serializers.ReadOnlyField(source="muallif.username") # faqat o'qish uchun
    kategoriya_nomi = serializers.ReadOnlyField(source="kategoriya.nomi")

    class Meta:
        model = Maqola
        fields = ["id", "sarlavha", "matn", "kategoriya", "kategoriya_nomi",
                  "muallif", "chop_etilgan", "korishlar", "yaratilgan"]
        read_only_fields = ["korishlar", "yaratilgan"]

    def validate_sarlavha(self, qiymat): # Forms'dagi clean_sarlavha kabi
        if len(qiymat.strip()) < 5:
            raise serializers.ValidationError("Sarlavha kamida 5 belgi bo'lsin")
        return qiymat.strip()
```

Shell'da sinab ko'ramiz

Python

```
from blog.models import Maqola
from blog.serializers import MaqolaSerializer

m = Maqola.objects.first()
s = MaqolaSerializer(m)
print(s.data)                                # obyekt → dict

s = MaqolaSerializer(Maqola.objects.all()[:2], many=True)  # ro'yxat uchun
many=True
print(s.data)

# Kelgan ma'lumotni tekshirish
s = MaqolaSerializer(data={"sarlavha": "Hi", "matn": "...", "kategoriya": 1})
print(s.is_valid())                          # False
print(s.errors)                             # {'sarlavha': ["Sarlavha
kamida 5 belgi bo'lsin"]}
```

Natija

```
{'id': 1, 'sarlavha': "Futbol bo'yicha g'alaba", 'matn': '...', 'kategoriya':
1, 'kategoriya_nomi': 'Sport',
  'muallif': 'admin', 'chop_etilgan': True, 'korishlar': 12, 'yaratilgan':
'2026-10-06T15:30:00+05:00'}
False
{'sarlavha': [ErrorDetail(string="Sarlavha kamida 5 belgi bo'lsin",
code='invalid')]}
```

API View'lar

1-usul: `@api_view` — funksiya asosida

blog/api_views.py (yangi fayl)

```
from django.shortcuts import get_object_or_404
from rest_framework import status
from rest_framework.decorators import api_view
from rest_framework.response import Response

from .models import Maqola
from .serializers import MaqolaSerializer

@api_view(["GET", "POST"])
def maqolalar_api(request):
    if request.method == "GET":
        maqolalar = Maqola.objects.filter(chop_etilgan=True)
        serializer = MaqolaSerializer(maqolalar, many=True)
        return Response(serializer.data) # 200 +

JSON

    # POST — yangi maqola
    serializer = MaqolaSerializer(data=request.data) #
    request.data — kelgan JSON
    if serializer.is_valid():
        serializer.save(muallif=request.user) #
    muallifni o'zimiz beramiz
        return Response(serializer.data, status=status.HTTP_201_CREATED)
    return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

@api_view(["GET"])
def maqola_api(request, id):
    maqola = get_object_or_404(Maqola, id=id, chop_etilgan=True)
    return Response(MaqolaSerializer(maqola).data)
```

2-usul: `APIView` — klass asosida

Python

```
from rest_framework.views import APIView

class MaqolalarAPI(APIView):
    def get(self, request):
        serializer = MaqolaSerializer(Maqola.objects.filter(chop_etilgan=True),
many=True)
        return Response(serializer.data)

    def post(self, request):
        serializer = MaqolaSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)      # xato bo'lsa –
avtomatik 400
        serializer.save(muallif=request.user)
        return Response(serializer.data, status=status.HTTP_201_CREATED)
```

blog/urls.py

```
from . import api_views

urlpatterns += [
    path("api/maqolalar/", api_views.maqolalar_api, name="api_maqolalar"),
    path("api/maqolalar/<int:id>/", api_views.maqola_api, name="api_maqola"),
]
```

Brauzerda ko'rinadigan API

Serverni ishga tushirib, `http://127.0.0.1:8000/api/maqolalar/` ni oching. DRF chiroyli, **interaktiv** sahifa ko'rsatadi: JSON javob, status kod va (tizimga kirgan bo'lsangiz) pastda POST qilish uchun forma. Bu API'ni sinashning eng oson usuli.

Boshqa usullar:

Terminal

```
# Terminal orqali (curl)
curl http://127.0.0.1:8000/api/maqolalar/
curl http://127.0.0.1:8000/api/maqolalar/1/
```

JavaScript'dan (brauzer konsoli yoki frontend)

```
fetch("http://127.0.0.1:8000/api/maqolalar/")
  .then(javob => javob.json())
  .then(maqolalar => {
    maqolalar.forEach(m => console.log(m.sarlavha, "-", m.muallif));
  });
```



Professional dasturchilar API'ni sinash uchun **Postman** yoki VS Code'dagi **Thunder Client** kengaytmasidan foydalanadi: istalgan metod, sarlavha va JSON tana bilan so'rov yuborish mumkin.

Amaliy mashg'ulot

- 1 DRF'ni o'rnatish va sozlash.
- 2 `serializers.py` yaratish, shell'da serializatsiya va validatsiyani sinab ko'ring.
- 3 `@api_view` bilan ikkita endpoint yaratish va brauzerda oching.
- 4 Admin orqali tizimga kiring, so'ng brauzerdagi API sahifasida POST forma orqali yangi maqola yaratish (JSON):

JSON

```
{
  "sarlavha": "API orqali yaratilgan maqola",
  "matn": "Bu maqola DRF yordamida qo'shildi!",
  "kategoriya": 1
}
```

Javob kodi `201` ekanini tekshiring. Sarlavhani 3 belgili qilib yuboring — `400` va xato xabari.

- 5 `/api/kategoriyalar/` endpointini yarating (`KategoriyaSerializer`), har bir kategoriyada maqolalar soni ham bo'lsin (Maslahat: `serializers.IntegerField(source="maqolalar.count", read_only=True)`).

Ko'p uchraydigan xatolar

- ❌ `Object of type Maqola is not JSON serializable` — obyekt serializer'dan o'tkazilmagan.
- ❌ Ro'yxat uchun `many=True` unutilgan — `AttributeError`.
- ❌ `serializer.data` ni `is_valid()` dan oldin o'qish (`data=` bilan yaratilganda).
- ❌ `"rest_framework"` `INSTALLED_APPS` ga qo'shilmagan — brauzerdagi API sahifasi shablon xatosi beradi.
- ❌ POST so'rov `403` qaytaradi — tizimga kirilmagan (`IsAuthenticatedOrReadOnly`).

O'zingizni tekshiring

? REST API'da yangi resurs yaratish uchun qaysi HTTP metod ishlatiladi?

- A. GET
- B. POST
- C. PUT
- D. DELETE

? Muvaffaqiyatli yaratilgan resurs uchun qaytariladigan status kod?

- A. 200 OK
- B. 204 No Content
- C. 201 Created
- D. 302 Found

? Serializer'ning vazifasi?

- A. Model obyektlarini JSON'ga va JSON'ni tekshirilgan obyektlarga aylantirish
- B. Bazaga ulanish
- C. HTML shablon yaratish
- D. URL'larni sozlash

? Qaysi URL REST qoidalariga mos?

- A. `/api/getAllMaqolalar/`
- B. `/api/maqola/ochirish/5/`
- C. `/api/maqolaYarat/`
- D. `/api/maqolalar/5/`

? Bir nechta obyektни serializatsiya qilish uchun qaysi parametr kerak?

- A. `all=True`
- B. `many=True`
- C. `list=True`
- D. `multiple=True`

? DRF view'ida kelgan JSON ma'lumot qayerda?

- A. `request.POST`
- B. `request.GET`
- C. `request.data`
- D. `request.json`



Uyga vazifa

1. Kutubxona loyihangiz (38-mavzu uyga vazifasi) uchun `KitobSerializer` va `/api/kitoblar/` (GET, POST) endpointini yarating.
2. Ochiq API'larni o'rganing: brauzerda `https://api.github.com/users/torvalds` ni oching. JSON tuzilmasini tahlil qilib, qaysi maydonlar borligini yozing.



Yodda tuting

- API — dasturlar orasidagi interfeys; veb-API JSON qaytaradi va ko'p mijozlarga xizmat qiladi.
- REST: resurs — URL (ot, ko'plik), amal — HTTP metod (GET/POST/PUT/PATCH/DELETE), natija — status kod.
- DRF: `pip install djangorestframework` + `INSTALLED_APPS`.
- `ModelSerializer` — model ↔ JSON; `validate_maydon()`; `many=True`; `is_valid()`, `save()`, `errors`.
- `@api_view` / `APIView` + `Response(data, status=...)`.

☐ Darsni o'qib bo'ldim

✓ Test javoblari

1. REST API'da yangi resurs yaratish uchun qaysi HTTP metod ishlatiladi? — **B) POST**
2. Muvaffaqiyatli yaratilgan resurs uchun qaytariladigan status kod? — **C) 201 Created**
3. Serializer'ning vazifasi? — **A) Model obyektlarini JSON'ga va JSON'ni tekshirilgan obyektlarga aylantirish**
4. Qaysi URL REST qoidalariga mos? — **D) /api/maqolalar/5/**
5. Bir nechta obyektни serializatsiya qilish uchun qaysi parametr kerak? — **B) many=True**
6. DRF view'ida kelgan JSON ma'lumot qayerda? — **C) request.data**